

Chapter 5: Architecture and Process Model of the RS-DRL-Based Self-Adaptive System (Alternative 2 - Clean Skeleton)

Thesis Documentation

February 8, 2026

Contents

5.1 Chapter Overview and Scope	2
5.1.1 Purpose of Chapter 5	2
5.1.2 Relationship to the Proposed Method	2
5.1.3 Focus on Architectural Structure and Process Flow	2
5.1.4 Chapter Organization	2
5.2 System-Level Self-Adaptive Architecture	3
5.2.1 High-Level System View	3
5.2.2 Separation Between Managed and Managing Systems	3
5.2.3 Continuous Adaptation Loop as Primary Abstraction	4
5.2.4 End-to-End Adaptation Flow	4
5.3 End-to-End Self-Adaptation Process Flow	4
5.3.1 Stage-Based Process Overview	4
5.3.2 Identification of Sensing, Decision-Making, Execution, and Learning	5
5.3.3 Distinction Between Runtime Adaptation and Asynchronous Learning	6
5.3.4 Reference Process Flow for Traceability	6
5.4 Mapping the Proposed Architecture to the MAPE-K Model	6
5.4.1 Conceptual Mapping to MAPE-K Components	6
5.4.2 Role of RS-DRL within MAPE-K Loop	6
5.4.3 Responsibilities of MAPE-K Components	7
5.4.4 Temporal Classification of Processes	7
5.5 Runtime Monitoring and State Construction	8
5.5.1 Runtime Data Collection	9
5.5.2 State Construction as Interface	9
5.5.3 Dissemination of State Information	10
5.5.4 Boundary Between Perception and Action	11
5.6 Runtime Adaptation Decision Process	11
5.6.1 Analysis and Adaptation Triggering	11
5.6.1.1 Evaluation of System State Against Goals	11
5.6.1.2 Detection of Goal Violations	11
5.6.1.3 Conditions for Adaptation Triggering	11
5.6.2 Planning and Adaptation Option Prediction	12
5.6.2.1 Retrieval of Learned RS-DRL Policy	12
5.6.2.2 Prediction of Suitable Adaptation Options	12
5.6.2.3 Decision-Making Based on Learned Estimates	12
5.6.2.4 Emphasis on Inference-Only Behavior	12

5.6.3	Adaptation Option Verification Under Uncertainty	13
5.6.3.1	Validation of Predicted Options	13
5.6.3.2	Use of Runtime Models	14
5.6.3.3	Generation of Quality Estimates	14
5.7	Execution and System Reconfiguration	15
5.7.1	Retrieval of Adaptation Option Specifications	16
5.7.2	Application of Selected Adaptation	16
5.7.3	Effect on System Behavior and Environment	16
5.7.4	Reintegration into Continuous Monitoring Loop	16
5.8	RS-DRL Feedback and Learning Overview	17
5.8.1	Role of Feedback in Linking Runtime and Learning	17
5.8.2	High-Level View of Reward Generation	17
5.8.3	Separation Between Feedback Generation and Learning	18
5.9	Asynchronous RS-DRL Learning and Policy Update	18
5.9.1	Overview of RS-DRL Learning Lifecycle	18
5.9.2	Experience Collection and Replay	19
5.9.3	Placement of Novel Reward-Shaping Mechanism	19
5.9.4	Model Update and Storage	19
5.10	Knowledge Management and Process Coordination	20
5.10.1	Role of Knowledge Repository as Shared Memory	21
5.10.2	Read-Write Interactions	21
5.10.3	Coordination of Heterogeneous Processes	21
5.10.4	Consistency and Separation of Concerns	21
5.11	Chapter Summary	21
5.11.1	Recap of Architectural Structure and Process Flow	21
5.11.2	Positioning Within Overall Thesis	22
5.11.3	Transition to Subsequent Chapters	22

5.1 Chapter Overview and Scope

This chapter presents the architecture and process model of the RS-DRL-based self-adaptive system. The chapter focuses on architectural structure and process flow, maintaining clear separation between system design (presented here) and algorithmic implementation details (presented in subsequent chapters or appendices).

5.1.1 Purpose of Chapter 5

Chapter 5 addresses the architectural organization of the proposed self-adaptive system, explaining how components are structured, how processes flow through the system, and how runtime adaptation is coordinated with asynchronous learning. The chapter emphasizes architectural responsibility and design rationale rather than algorithmic mechanics or experimental results.

5.1.2 Relationship to the Proposed Method

This chapter corresponds to the "Proposed Method" section of the thesis, detailing the architectural realization of the RS-DRL approach. While the article presents RS-DRL as an integrated method, this chapter decomposes the architecture into understandable components and processes, enabling readers to comprehend system organization before examining implementation details or evaluation results.

5.1.3 Focus on Architectural Structure and Process Flow

The chapter maintains strict focus on:

- **Structural organization:** How components are arranged and interact
- **Process flow:** How data and control flow through the system
- **Design rationale:** Why architectural decisions were made

The chapter explicitly excludes:

- Experimental results and evaluation (covered in Chapter 6)
- Deep algorithmic mathematics (covered in appendices)
- Background concepts (covered in earlier chapters)

5.1.4 Chapter Organization

The chapter is organized as follows:

- Section 5.2: System-level architectural view
- Section 5.3: End-to-end adaptation process flow
- Section 5.4: Mapping to MAPE-K reference model
- Section 5.5: Runtime monitoring and state construction
- Section 5.6: Runtime adaptation decision process
- Section 5.7: Execution and system reconfiguration
- Section 5.8: RS-DRL feedback and learning overview
- Section 5.9: Asynchronous RS-DRL learning and policy update
- Section 5.10: Knowledge management and process coordination
- Section 5.11: Chapter summary

5.2 System-Level Self-Adaptive Architecture

This section provides a high-level view of the RS-DRL-based self-adaptive system, establishing the architectural foundation for subsequent detailed descriptions.

5.2.1 High-Level System View

The RS-DRL-based self-adaptive system consists of two primary architectural components:

- **Managed System:** Executes application logic and generates runtime data
- **Managing System:** Implements adaptation logic using MAPE-K enhanced with RS-DRL

This architectural separation reflects a fundamental design principle: the managed system focuses on domain-specific functionality, while the managing system focuses on adaptation control.

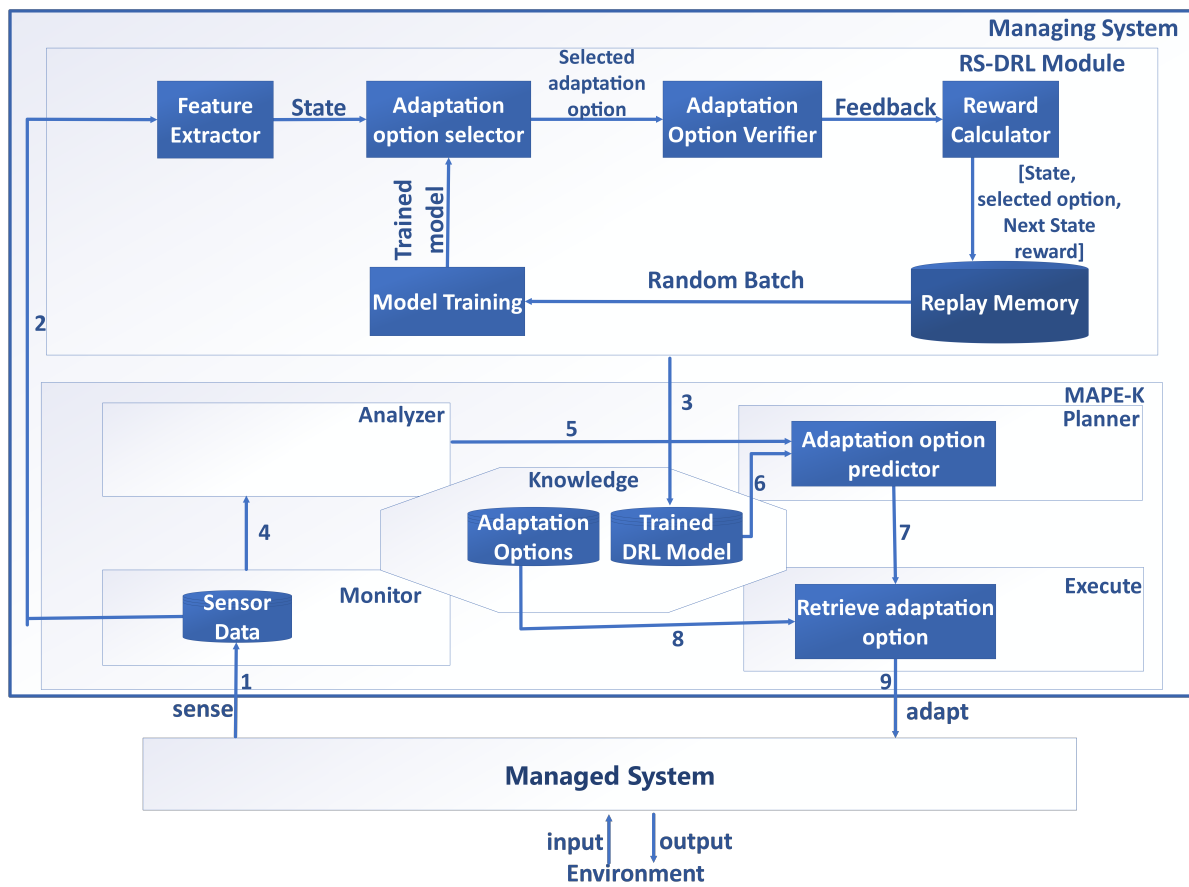


Figure 1: RS-DRL integration architecture showing the separation between managed and managing systems, with RS-DRL enhancing the MAPE-K adaptation framework.

5.2.2 Separation Between Managed and Managing Systems

The interface between managed and managing systems is defined by:

- **Upward flow:** Runtime data streams from managed to managing system
- **Downward flow:** Adaptation actions from managing to managed system

This clean interface enables the managing system to adapt the managed system without requiring modifications to application logic.

5.2.3 Continuous Adaptation Loop as Primary Abstraction

At the highest level, the system operates as a continuous adaptation loop:

1. **Sense:** Monitor system state and environmental conditions
2. **Decide:** Evaluate adaptation needs and select appropriate actions
3. **Act:** Execute selected adaptations
4. **Learn:** Refine adaptation policies based on outcomes

This loop executes continuously, enabling the system to maintain adaptation goals under changing conditions.

5.2.4 End-to-End Adaptation Flow

The continuous adaptation loop is realized through an end-to-end adaptation flow that will be detailed in Section 5.3. This flow provides traceability throughout the chapter, enabling readers to understand how high-level architectural concepts map to specific system behaviors.

5.3 End-to-End Self-Adaptation Process Flow

This section presents a stage-based overview of the complete adaptation cycle, providing the reference process flow used for traceability in subsequent sections.

5.3.1 Stage-Based Process Overview

The adaptation cycle is decomposed into nine stages that capture the complete flow from sensing to learning:

1. **Stage 1:** Runtime data collection from managed system
2. **Stage 2:** State forwarding to RS-DRL module
3. **Stage 3:** Asynchronous model training and storage
4. **Stage 4:** State forwarding to Analyzer
5. **Stage 5:** Adaptation triggering
6. **Stage 6:** Trained model retrieval
7. **Stage 7:** Adaptation option prediction
8. **Stage 8:** Adaptation option retrieval
9. **Stage 9:** Adaptation application

These stages provide a common vocabulary for discussing system behavior throughout the chapter.

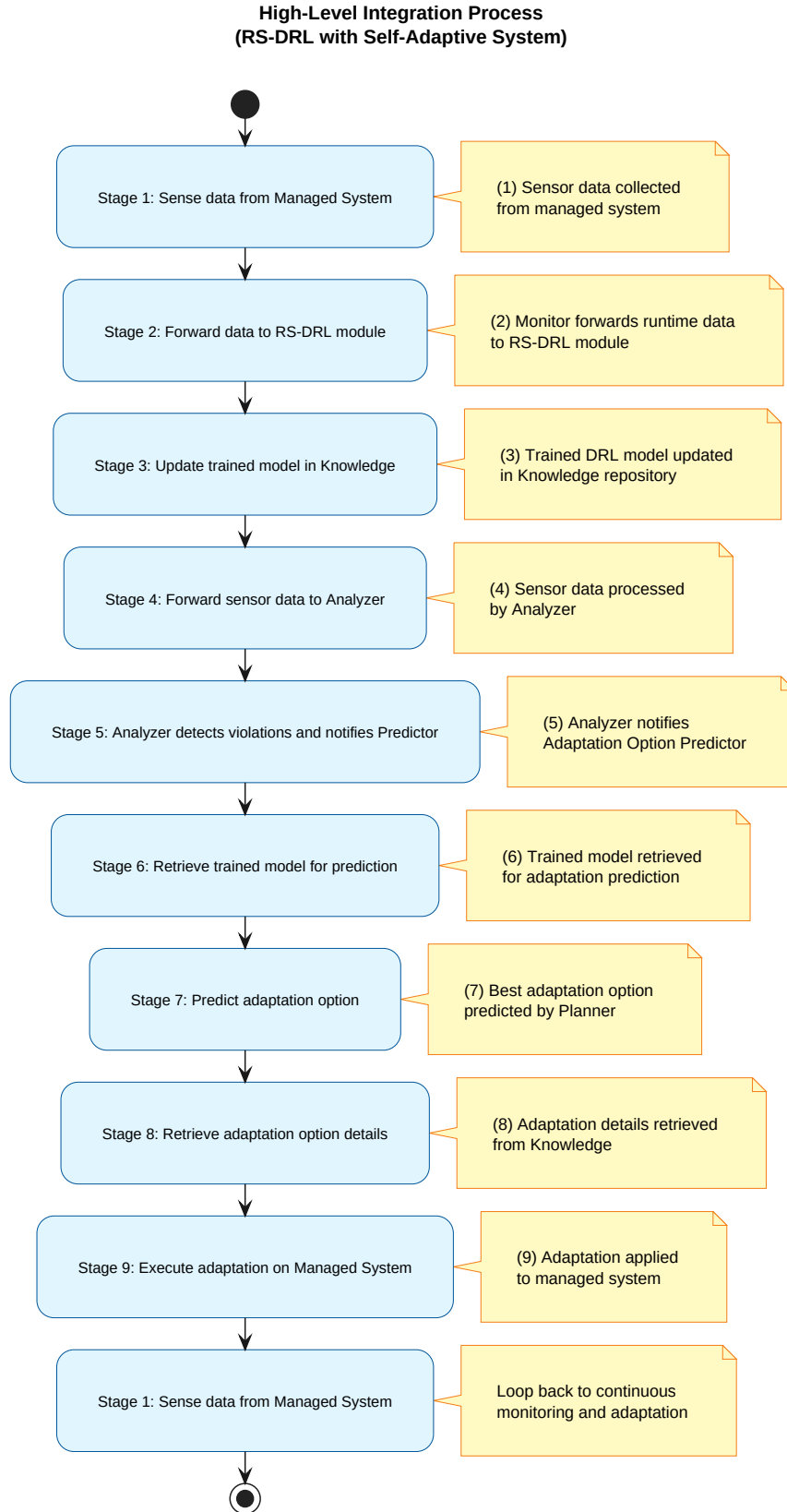


Figure 2: End-to-end self-adaptation process flow showing the nine stages from sensing through learning.

5.3.2 Identification of Sensing, Decision-Making, Execution, and Learning

The nine stages can be grouped into four primary process categories:

- **Sensing** (Stages 1, 2, 4): Monitoring and state construction
- **Decision-Making** (Stages 5, 6, 7): Analysis, planning, and prediction

- **Execution** (Stages 8, 9): Adaptation enactment
- **Learning** (Stages 2, 3): Model training and update

This categorization helps readers understand the functional organization of the system.

5.3.3 Distinction Between Runtime Adaptation and Asynchronous Learning

A critical architectural design decision is the temporal separation between runtime adaptation (Stages 1, 4–9) and asynchronous learning (Stages 2–3):

- **Runtime adaptation:** Operates synchronously with system execution, using pre-trained models
- **Asynchronous learning:** Operates in background, refining models without disrupting runtime

This separation ensures runtime stability while enabling continuous policy improvement.

5.3.4 Reference Process Flow for Traceability

All subsequent sections reference these nine stages when describing component behaviors and interactions. This traceability mechanism enables readers to understand how detailed descriptions relate to the overall system flow.

5.4 Mapping the Proposed Architecture to the MAPE-K Model

This section provides a conceptual mapping of the proposed system to the MAPE-K architectural pattern, establishing connections to established self-adaptation frameworks.

5.4.1 Conceptual Mapping to MAPE-K Components

The RS-DRL-based architecture maps to MAPE-K components as follows:

- **Monitor:** Observes managed system and constructs state representations (Stages 1, 2, 4)
- **Analyze:** Evaluates state against goals and triggers adaptation (Stages 4, 5)
- **Plan:** Selects adaptation options using RS-DRL policies (Stages 5, 6, 7)
- **Execute:** Applies selected adaptations to managed system (Stages 8, 9)
- **Knowledge:** Stores models, options, and state history (Stages 3, 6, 8)

5.4.2 Role of RS-DRL within MAPE-K Loop

RS-DRL is positioned as an enhancement to the **Plan** phase, providing learned intelligence for adaptation option selection. The RS-DRL module does not replace MAPE-K components but rather enriches planning capability with data-driven decision-making.

The RS-DRL module spans multiple MAPE-K phases:

- Receives input from Monitor (Stage 2)
- Supports decision-making in Plan (Stages 6–7)
- Stores learned knowledge (Stage 3)

This cross-cutting nature reflects RS-DRL’s role as an intelligent decision service rather than a standalone component.

5.4.3 Responsibilities of MAPE-K Components

Each MAPE-K component has clearly defined responsibilities:

- **Monitor:** Perception and state abstraction
- **Analyze:** Goal evaluation and adaptation triggering
- **Plan:** Decision-making using learned policies
- **Execute:** Adaptation enactment
- **Knowledge:** Coordination and persistence

These responsibilities maintain clear separation of concerns across the architecture.

5.4.4 Temporal Classification of Processes

Processes are classified by temporal characteristics:

Process	Temporal Nature
Monitoring	Continuous
Analysis	Event-driven
Planning	Event-driven
Execution	Event-driven
Learning	Asynchronous

Table 1: Temporal classification of architectural processes.

This classification justifies architectural decisions regarding process separation and coordination.

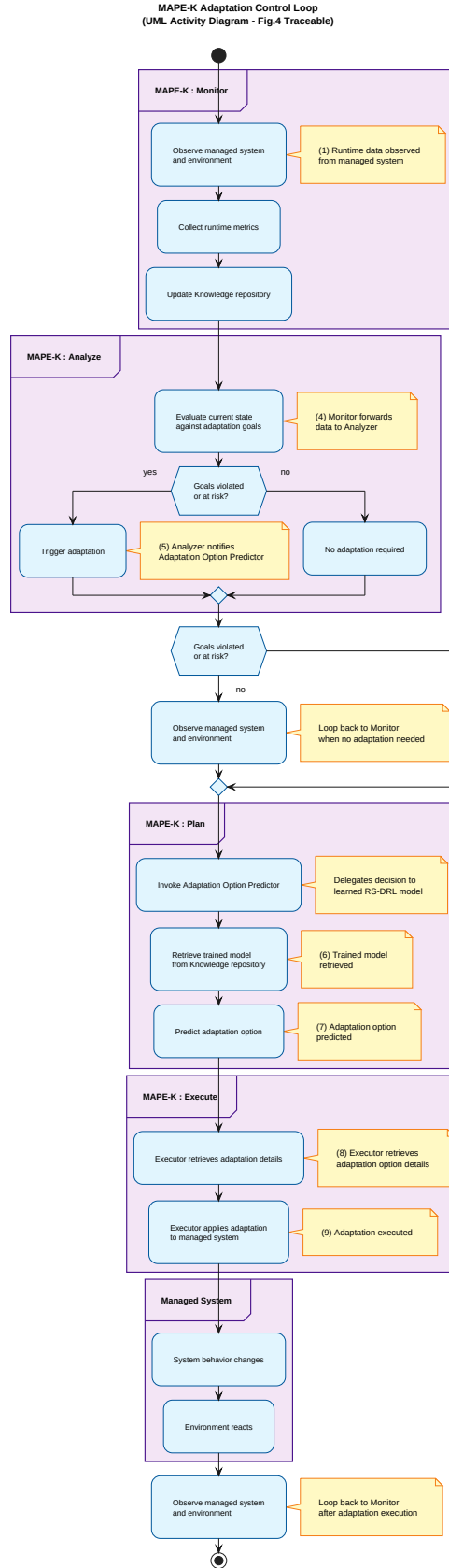


Figure 3: MAPE-K validation view showing how RS-DRL enhances the planning phase within the adaptation control loop.

5.5 Runtime Monitoring and State Construction

This section describes the monitoring process that transforms raw runtime data into decision-ready state representations.

5.5.1 Runtime Data Collection

The monitoring process begins with runtime data collection from the managed system and environment (Stage 1). Data sources include:

- System performance metrics (latency, throughput)
- Resource utilization (CPU, memory, network)
- Environmental conditions (workload, request patterns)

5.5.2 State Construction as Interface

Raw data is transformed into a validated state vector that serves as the interface between monitoring and decision-making. The state construction process:

1. Preprocesses raw data (filtering, normalization)
2. Extracts goal-relevant features
3. Validates state vector consistency

The state vector $s_t \in \mathbb{R}^d$ provides a compact, consistent representation used by all downstream components.

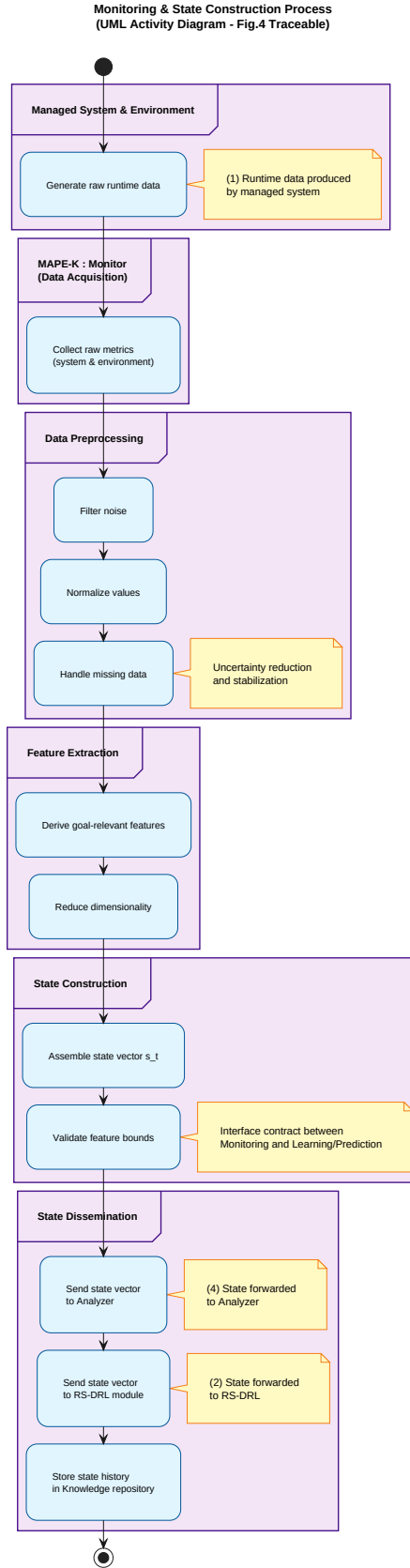


Figure 4: Monitoring and state construction process transforming raw data into validated state vectors.

5.5.3 Dissemination of State Information

Once constructed, the state vector is disseminated to:

- **Analyzer** (Stage 4): For goal evaluation
- **RS-DRL module** (Stage 2): For learning or inference

- **Knowledge repository:** For historical reference

5.5.4 Boundary Between Perception and Action

The monitoring process ends once a validated state vector exists and has been disseminated. This boundary ensures clear separation between perception (monitoring) and action (planning and execution).

5.6 Runtime Adaptation Decision Process

This section describes the runtime decision-making process, encompassing analysis, planning, and verification.

5.6.1 Analysis and Adaptation Triggering

5.6.1.1 Evaluation of System State Against Goals

The Analyzer receives state vectors from the Monitor (Stage 4) and evaluates whether the current system state satisfies adaptation goals. Goals may include:

- Performance targets (response time, throughput)
- Resource constraints (energy, memory)
- Quality-of-service requirements

5.6.1.2 Detection of Goal Violations

When the Analyzer detects that adaptation goals are violated or at risk, it triggers the planning phase (Stage 5). The triggering decision is based on:

- Current goal satisfaction status
- Predicted future violations
- Risk assessment under uncertainty

5.6.1.3 Conditions for Adaptation Triggering

Adaptation is triggered when:

1. Goals are currently violated
2. Goals are predicted to be violated soon
3. System state indicates increased risk

If no triggering condition is met, the system continues normal operation without adaptation.

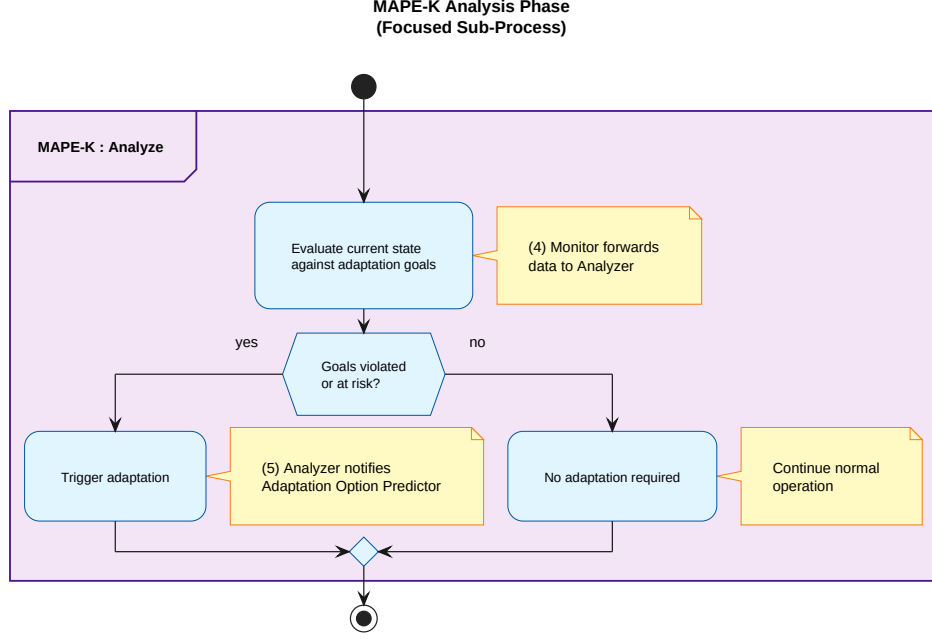


Figure 5: Analysis and adaptation triggering process evaluating system state against goals.

5.6.2 Planning and Adaptation Option Prediction

5.6.2.1 Retrieval of Learned RS-DRL Policy

Upon receiving the adaptation trigger (Stage 5), the planning process retrieves the trained RS-DRL model from the Knowledge repository (Stage 6). The model encodes learned adaptation policies that map states to adaptation options.

5.6.2.2 Prediction of Suitable Adaptation Options

The Adaptation Option Predictor evaluates all available adaptation options using the retrieved model. For each option a , the predictor estimates its expected long-term value:

$$Q(s_t, a; \theta) \rightarrow \text{expected value of option } a \text{ in state } s_t$$

where θ represents the learned model parameters.

5.6.2.3 Decision-Making Based on Learned Estimates

The predictor selects the adaptation option with the highest estimated value:

$$a^* = \arg \max_{a \in \mathcal{A}} Q(s_t, a; \theta)$$

This selection leverages learned knowledge to make informed decisions under uncertainty.

5.6.2.4 Emphasis on Inference-Only Behavior

At runtime, the planning process performs **pure inference**—it uses pre-trained models to evaluate options but performs no learning, no replay memory sampling, and no policy updates. This ensures predictable response times and runtime stability.

Adaptation Option Prediction Process
(Runtime Decision Path - Stages 5-7 traceable to Figure 2)

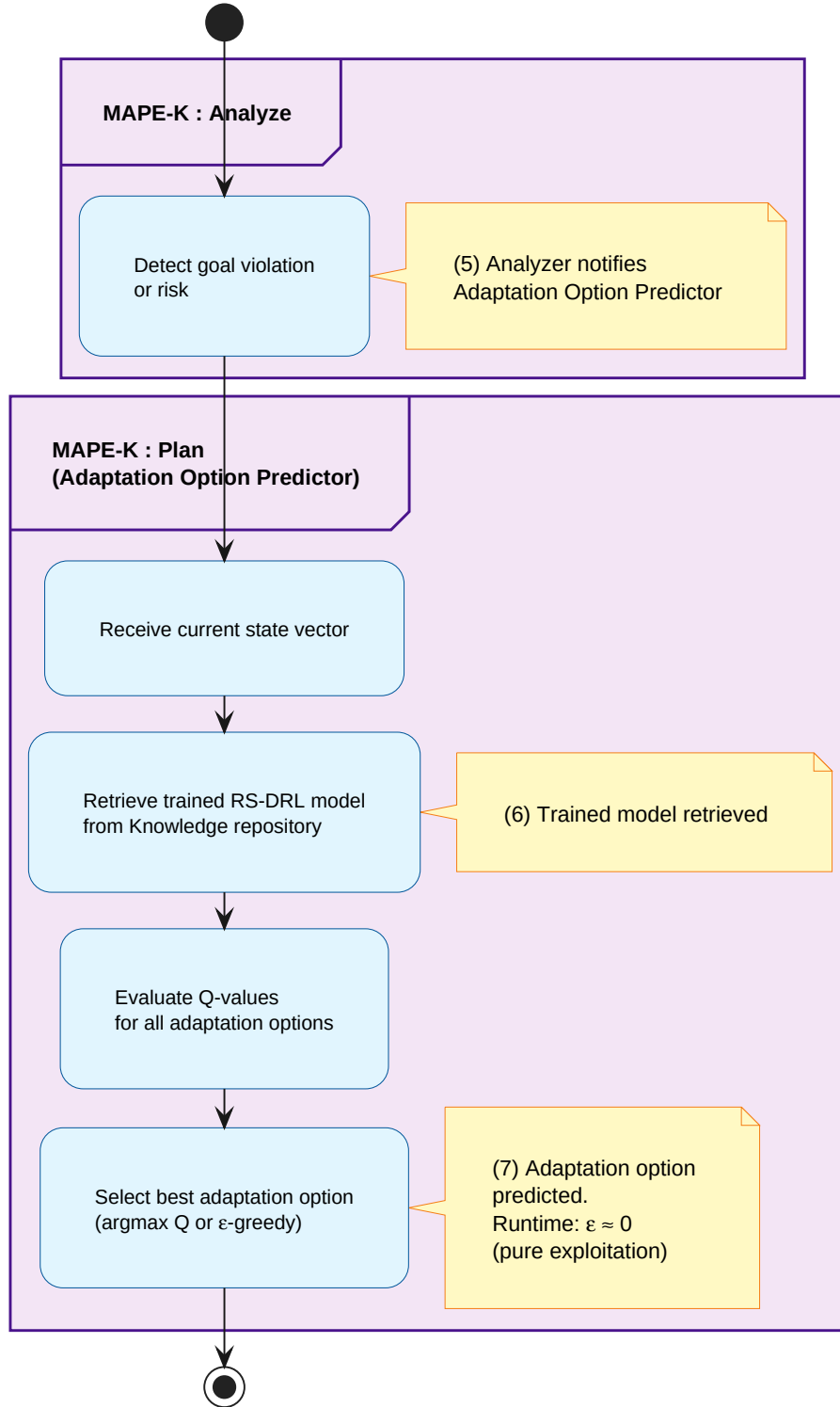


Figure 6: Planning and adaptation option prediction using learned RS-DRL policies.

5.6.3 Adaptation Option Verification Under Uncertainty

5.6.3.1 Validation of Predicted Options

The predicted adaptation option undergoes verification to provide decision justification under uncertainty. Verification evaluates whether the predicted option is likely to satisfy adaptation goals given current uncer-

tainties.

5.6.3.2 Use of Runtime Models

Verification employs runtime models that abstract system behavior and environmental uncertainties. These models are parameterized with:

- Current system state
- Candidate adaptation option
- Uncertainty parameters

5.6.3.3 Generation of Quality Estimates

Through statistical model checking (using UPPAAL-SMC), verification generates quality estimates including:

- Goal satisfaction probabilities
- Expected quality-of-service metrics
- Confidence intervals for predictions

These estimates justify adaptation decisions, providing trustworthiness for learning-based choices.

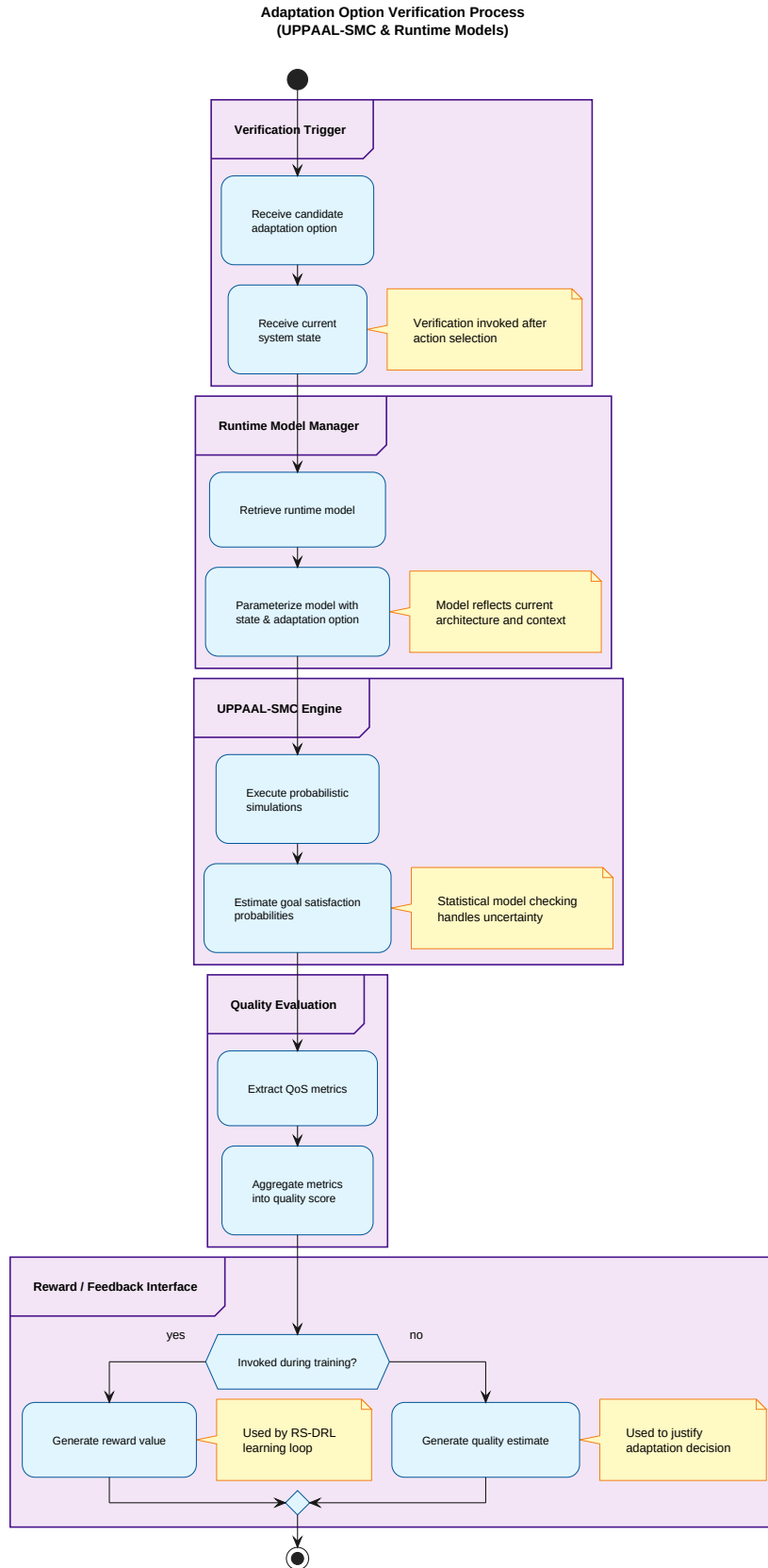


Figure 7: Adaptation option verification using runtime models and statistical model checking.

5.7 Execution and System Reconfiguration

This section describes how selected adaptations are applied to the managed system.

5.7.1 Retrieval of Adaptation Option Specifications

The Executor retrieves detailed specifications for the selected adaptation option from the Knowledge repository (Stage 8). Specifications define:

- Configuration parameters to modify
- Resource allocations to adjust
- Component interactions to reconfigure

5.7.2 Application of Selected Adaptation

The Executor translates the adaptation specification into concrete system modifications (Stage 9). The application process:

1. Validates adaptation safety
2. Applies configuration changes
3. Verifies successful application

5.7.3 Effect on System Behavior and Environment

Adaptation execution alters system behavior, affecting:

- Performance characteristics
- Resource consumption
- Quality-of-service attributes

These effects are observed by the monitoring process in subsequent cycles.

5.7.4 Reintegration into Continuous Monitoring Loop

After adaptation execution, the system returns to the monitoring phase (Stage 1), closing the adaptation loop. The continuous cycle ensures perpetual adaptation to changing conditions.

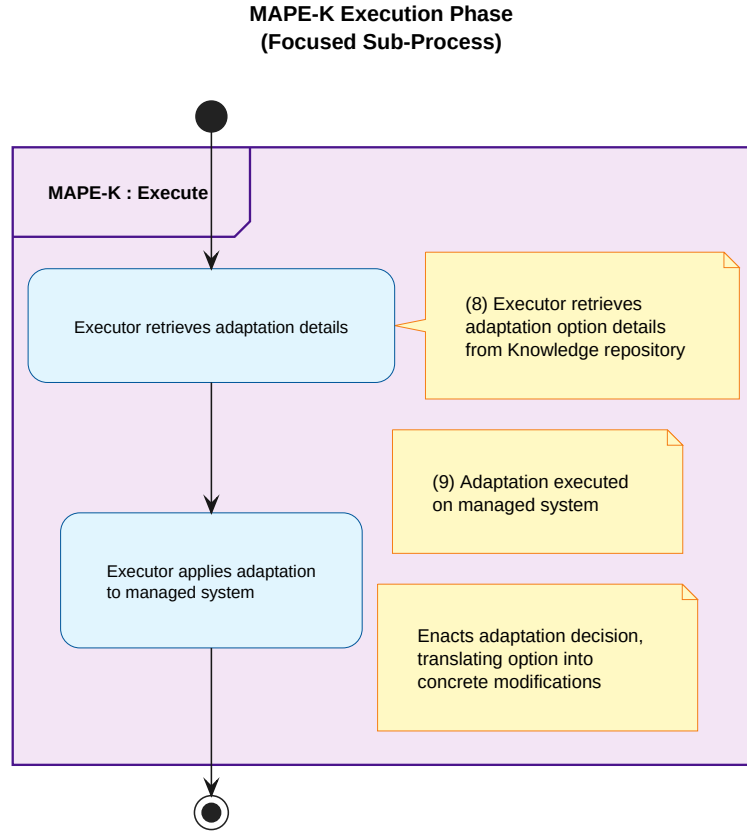


Figure 8: Execution and system reconfiguration process applying selected adaptations.

5.8 RS-DRL Feedback and Learning Overview

This section provides a high-level view of how runtime adaptation outcomes inform learning.

5.8.1 Role of Feedback in Linking Runtime and Learning

Feedback serves as the bridge between runtime adaptation and learning, enabling the system to improve policies based on adaptation outcomes. The feedback mechanism:

1. Observes adaptation effects
2. Evaluates outcome quality
3. Generates learning signals

5.8.2 High-Level View of Reward Generation

Reward signals are generated from verification outcomes, capturing:

- Quality-of-service metrics
- Goal satisfaction indicators
- Adaptation effectiveness measures

These rewards guide the learning process toward policies that achieve adaptation goals.

5.8.3 Separation Between Feedback Generation and Learning

The architecture maintains clear separation between:

- **Feedback generation:** Transforms outcomes into learning signals (this section)
- **Policy learning:** Consumes signals to update models (Section 5.9)

This separation enables independent evolution of feedback mechanisms and learning algorithms.

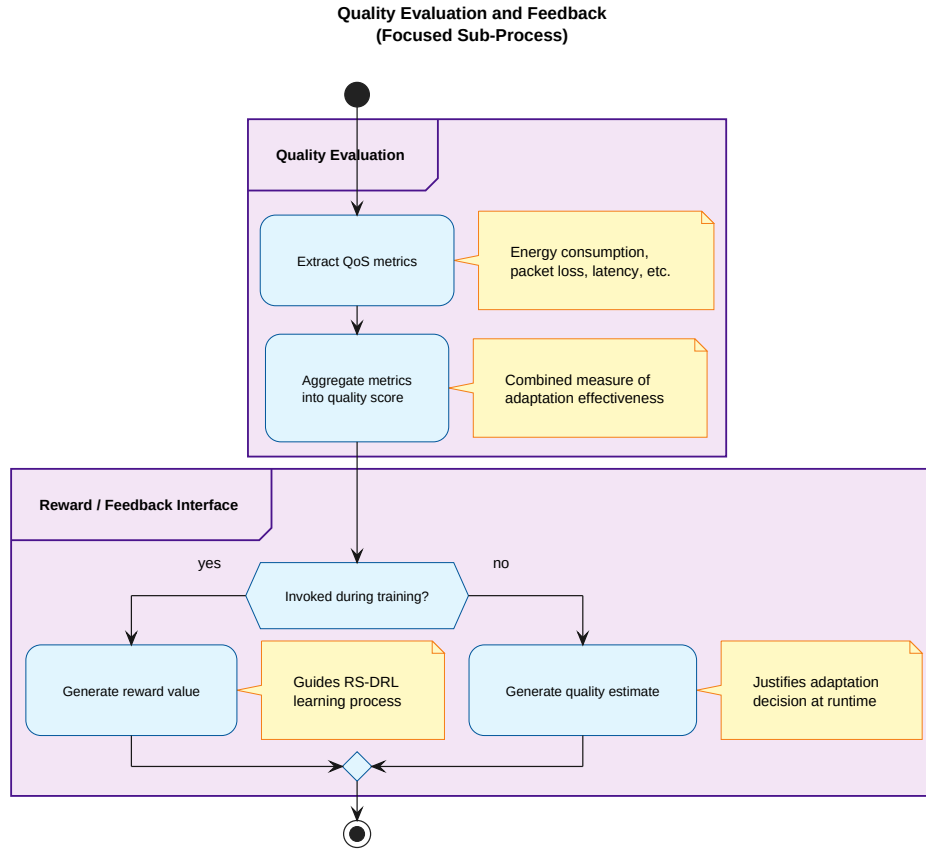


Figure 9: Feedback generation transforming adaptation outcomes into learning signals.

5.9 Asynchronous RS-DRL Learning and Policy Update

This section provides an overview of the RS-DRL learning process that refines adaptation policies.

5.9.1 Overview of RS-DRL Learning Lifecycle

The learning lifecycle operates asynchronously with runtime adaptation:

1. **Experience collection:** Gather state-action-reward-state tuples
2. **Experience storage:** Maintain replay memory of past experiences
3. **Model training:** Update policy using sampled experiences
4. **Model storage:** Persist updated model for runtime use (Stage 3)

This lifecycle repeats over multiple episodes, progressively improving adaptation policies.

5.9.2 Experience Collection and Replay

During training episodes, the system collects experiences through environment interaction:

$$\langle s_t, a_t, r_t, s_{t+1} \rangle$$

These experiences are stored in replay memory, enabling:

- Sample-efficient learning
- Breaking temporal correlations
- Learning from rare events

5.9.3 Placement of Novel Reward-Shaping Mechanism

The **reward-shaping mechanism** represents the key innovation of the RS-DRL approach. This mechanism:

- Selectively reshapes failed experiences
- Treats certain failures as optimistic successes
- Accelerates learning in heterogeneous environments
- Promotes policy generalization

The reward-shaping process operates during model training, transforming the experience distribution to facilitate faster convergence while maintaining long-term policy quality.

5.9.4 Model Update and Storage

Model training updates the deep Q-network parameters through:

1. Sampling minibatches from replay memory
2. Computing temporal difference targets
3. Performing gradient descent on loss function
4. Updating target network periodically

The updated model is stored in the Knowledge repository (Stage 3), making it available for future runtime adaptation (Stage 6).

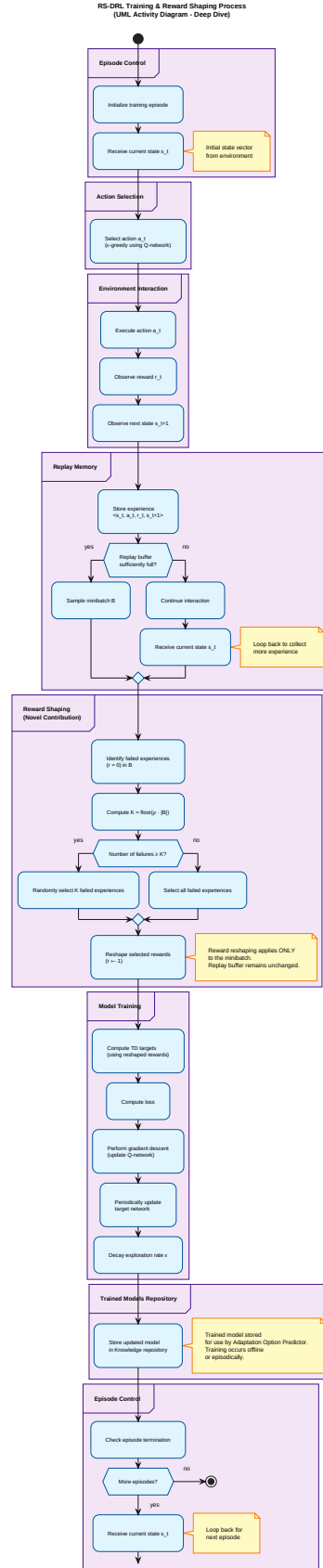


Figure 10: Asynchronous RS-DRL learning process including experience replay and novel reward-shaping mechanism.

5.10 Knowledge Management and Process Coordination

This section describes how the Knowledge repository coordinates processes with different temporal characteristics.

5.10.1 Role of Knowledge Repository as Shared Memory

The Knowledge repository serves as shared system memory, storing:

- **Trained models:** RS-DRL policies learned through training
- **Adaptation options:** Specifications for available adaptations
- **State history:** Historical state vectors for reference

5.10.2 Read–Write Interactions

Knowledge coordination occurs through well-defined read-write contracts:

- **Monitor writes:** State vectors for historical reference (Stage 1)
- **Learning writes:** Trained models after training episodes (Stage 3)
- **Planning reads:** Trained models for decision-making (Stage 6)
- **Execution reads:** Adaptation specifications for enactment (Stage 8)

5.10.3 Coordination of Heterogeneous Processes

The Knowledge repository enables processes with different temporal characteristics to synchronize:

- **Continuous monitoring:** Writes state continuously
- **Event-driven planning:** Reads models when triggered
- **Asynchronous learning:** Writes models after episodes

This coordination maintains consistency without tight coupling between processes.

5.10.4 Consistency and Separation of Concerns

Knowledge management ensures:

- **Consistency:** All components operate on coherent information
- **Separation:** Clear boundaries between component responsibilities
- **Persistence:** Critical information survives component failures

5.11 Chapter Summary

This chapter presented the architecture and process model of the RS-DRL-based self-adaptive system.

5.11.1 Recap of Architectural Structure and Process Flow

The chapter established:

- **System architecture:** Separation between managed and managing systems
- **Process flow:** Nine-stage adaptation cycle from sensing through learning
- **MAPE-K mapping:** Integration with established self-adaptation patterns
- **Component responsibilities:** Clear separation of monitoring, analysis, planning, execution, and learning
- **Temporal organization:** Distinction between runtime adaptation and asynchronous learning

5.11.2 Positioning Within Overall Thesis

This chapter provides the architectural foundation for understanding the RS-DRL approach. The chapter:

- Builds on background concepts (Chapters 1–4)
- Precedes experimental evaluation (Chapter 6)
- Establishes terminology and concepts used throughout remaining chapters

5.11.3 Transition to Subsequent Chapters

Having established the architectural structure and process model, the thesis proceeds to:

- **Chapter 6:** Experimental evaluation demonstrating effectiveness
- **Chapter 7:** Discussion of results and implications
- **Chapter 8:** Conclusions and future work

The architectural foundation provided here enables systematic evaluation and interpretation of experimental results in subsequent chapters.